

Profiling GPU codes with NVIDIA NSight

Trung Nguyen

22 June 2026

Frequently asked questions

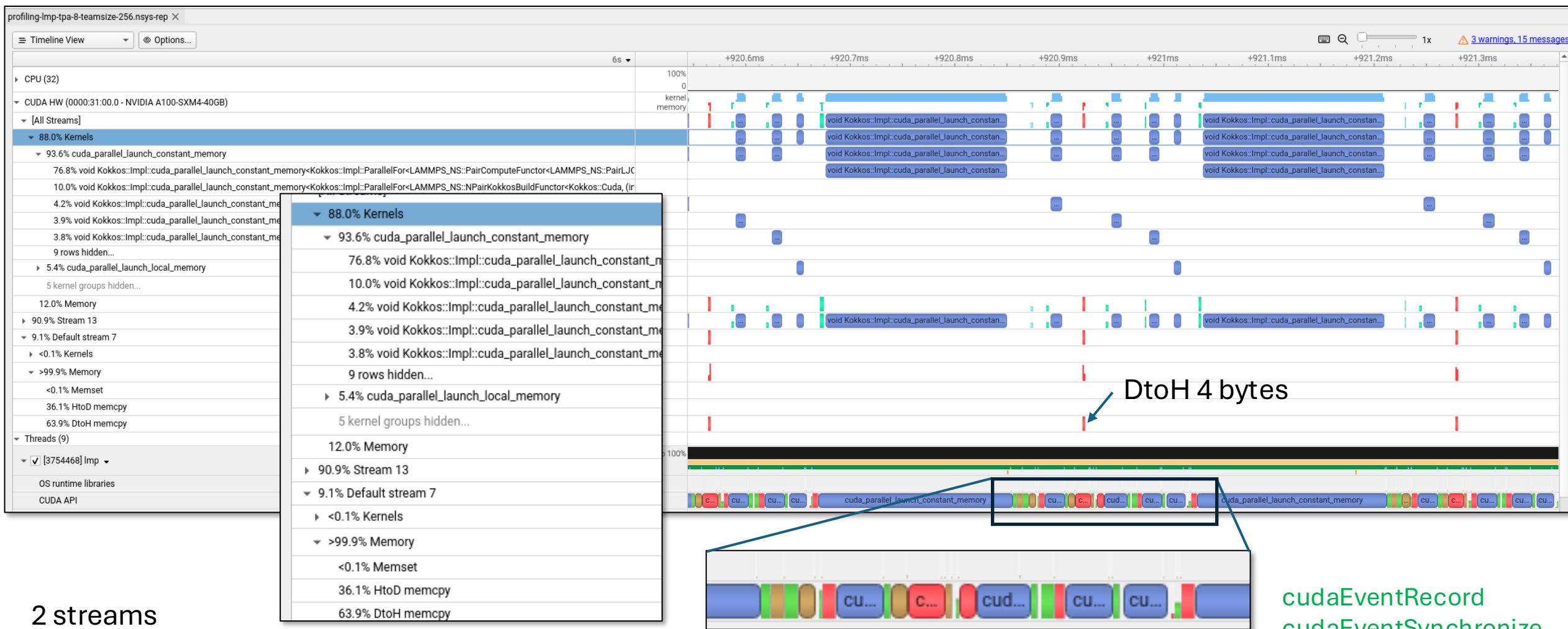
1. Why does the performance of my code vary
 - with training parameters (batch sizes, precisions)?
 - with different GPU architectures?
 - with different HPC clusters?
2. Can I improve the performance of my code?
 - If so, which part(s) of the code should be improved?
 - what are the best performance I can achieve for a given GPU architecture?

Profilers are tools for identifying performance bottlenecks: CPU and GPU usage, memory allocation, and execution patterns.

Profiling tools for GPU codes

- TAU (<https://www.cs.uoregon.edu/research/tau/home.php>) (CUDA, OpenACC)
- HPCToolkit (<https://hpctoolkit.org/>) (CUDA)
- Intel VTune Profiler (OpenCL, SYCL, oneAPI)
- AMD rocprofiler (ROCm HIP)
- scalene (<https://github.com/plasma-umass/scalene>) (CUDA)
 - Dr. Rahul Mehta (RCC)
- **NVIDIA NSight Systems and NSight Compute** (CUDA)
 - <https://docs.nvidia.com/nsight-systems/UserGuide/index.html>
 - <https://docs.nvidia.com/nsight-compute/NsightCompute/index.html>

My related work: Profiling Molecular Dynamics codes (MPI + GPU) with NVIDIA Nsight Systems



2 streams

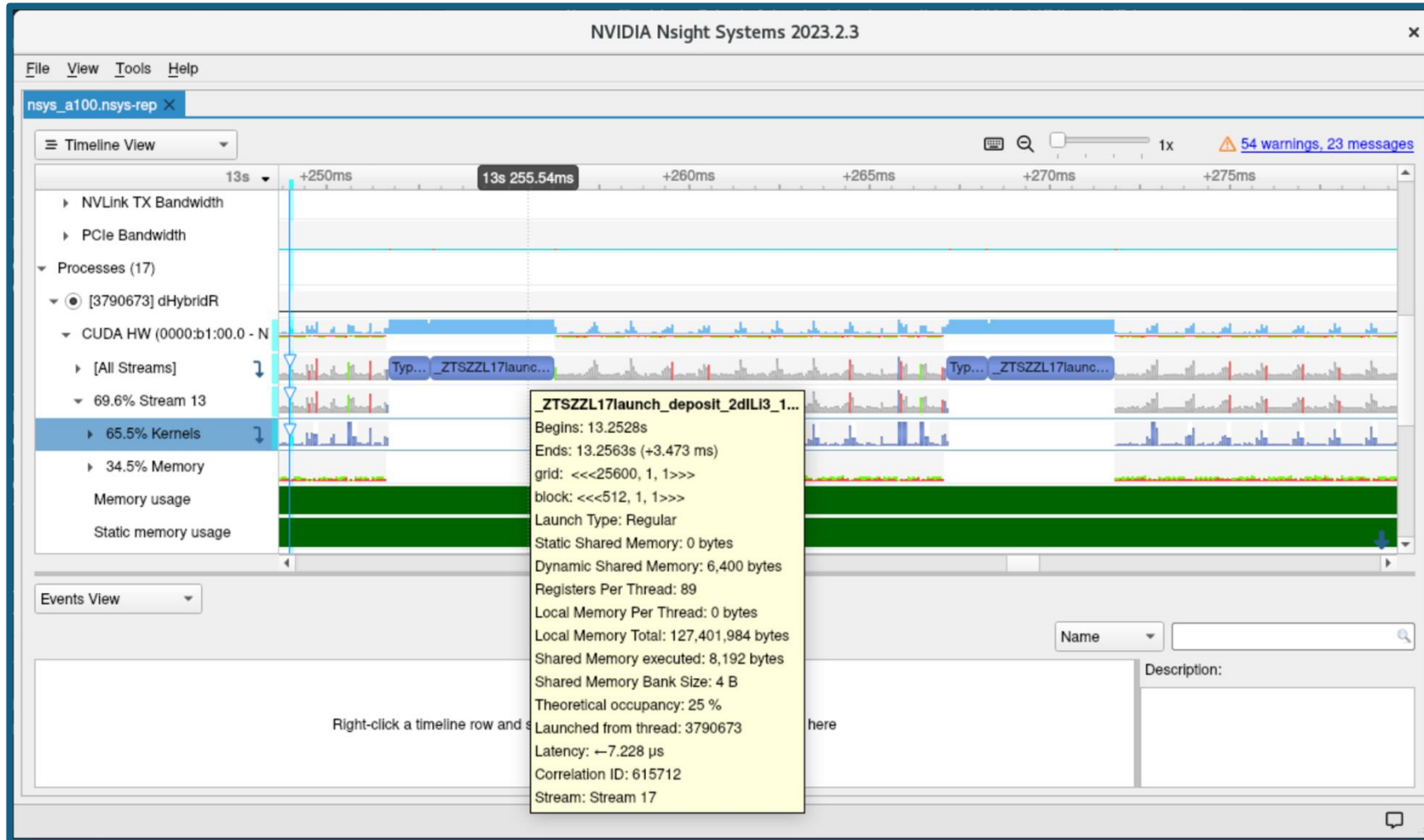
88% kernels (computation)

12% memory

LAMMPS and HOOMD-Blue

cudaEventRecord
cudaEventSynchronize
cudaDeviceSynchronize
cudaMemcpy

Profiling the plasma simulation code dHybridR on Midway GPUs (PI Caprioli)



Goals of this session

- How GPUs work and best-case scenarios for GPU acceleration
- How to use NVIDIA NSight Systems and Compute to analyze the performance of CUDA codes
- Recommended practices

On Midway3

- Sign in to the Open OnDemand portal: <https://midway3-ondemand.rcc.uchicago.edu/> under cVPN or on-campus network
- Launch the Midway3 Desktop App
- On the compute node, open a Terminal app:
`module load cuda/12.2 python/miniforge-25.3.0`
- Clone the repo
`git clone https://github.com/ndtrung81/cuda-profiling`

or go to

`cd /project/rcc/shared/cuda-profiling`

Applications (e.g. Python, C/C++ and Fortran-compiled programs)

Libs, Apps & Frameworks

CUDA Math Libs

cupy

cuDF

cuML

PyTorch

Tensorflow

JAX

CUDA

Runtime

Developer Tools

Compiler

Debugger

Profilers

Driver API

Context Mgt

Memory Mgt

Graphics

Communication

GPU Driver

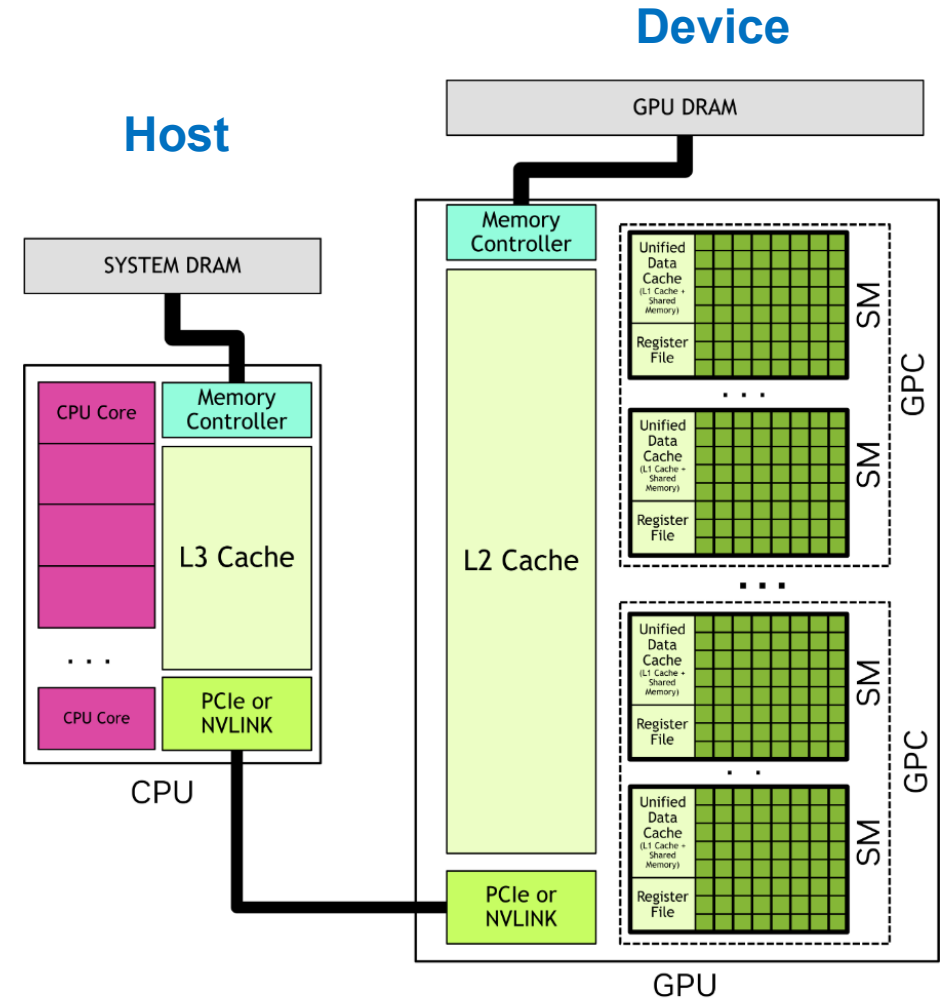
Operating System

GPUs

Heterogeneous systems: Separate memory spaces & compute units

Applications when run on the CPU side controls memory allocation and data transfers and launches computation tasks on the GPU:

1. CPU processes can run concurrently with the GPU.
2. Multiple tasks can be launched on the GPU at the same time.
 - Compute
 - Mem copies
3. Automatic scalability: just need to run on more powerful GPUs.



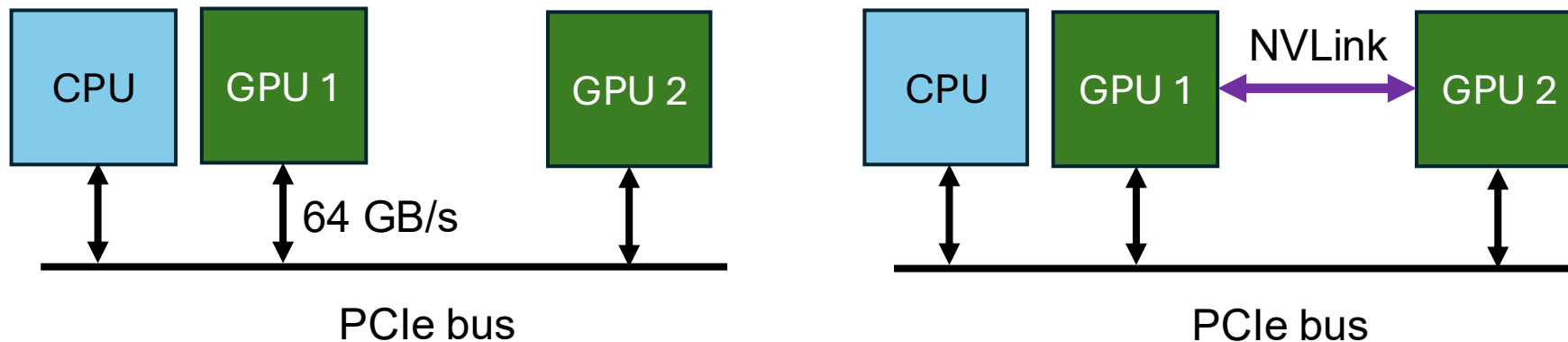
Data center GPUs vs Consumer GPUs

- For example, A100 PCIe vs Quadro RTX A6000, both based on the Ampere architecture

	Data Center GPUs	Consumer GPUs
Purposes	Computing	Rendering and computing
Usage and power consumption	Constant load 24/7, power-plant-like power management	May vary widely, household-like power management
Capacity	More SMs, mem bandwidth, and FP64 support	Ray tracing (RT) cores
Performance per \$	Application dependent	Application dependent
Output	N/A	Display output (HDMI, DisplayPort)
Fault tolerance	Mem error correction (ECC) req	Not req
Cooling	Passive (no fan)	Active (with fan or heatsink)
Cost	\$\$\$\$	\$

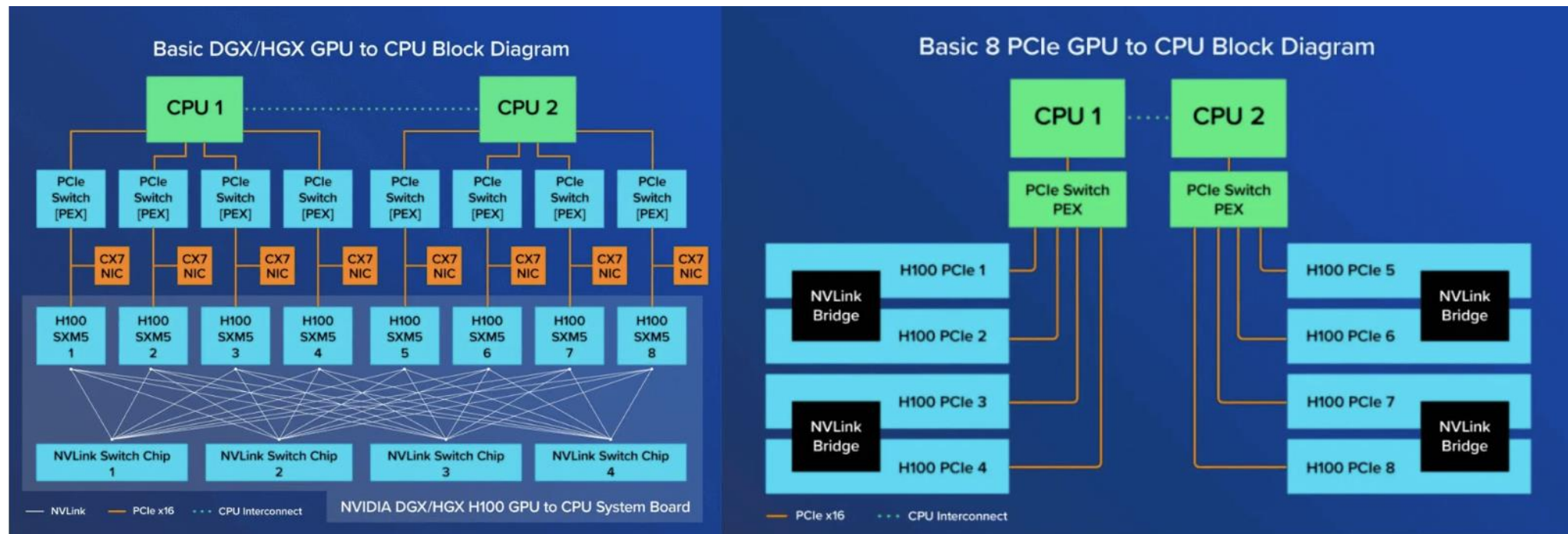
PCIe-only vs With NVLink

- GPU-GPU connection: controls how data is transferred between the GPUs
 - PCIe-only: through the mainboard sockets (the most popular configuration)
 - With NVLink: through physical bridges (non-DGX) or switches (DGX)
 - 900 GB/s (v4.0, H100s), 1.8 TB/s (v5.0, Blackwell)
 - used by CUDA API that use the underlying NVIDIA NCCL for device-to-device (p2p) mem copies



PCIe interface vs SXM interface

- PCIe: GPUs are connected to mainboard at PCIe slots
 - only pairs of GPUs can be connected by NVLink
- SXM (Server PCIe Express Module): available in DGX/HGX servers
 - GPUs can be fully connected via NVLink Switch (8-way)



`nvidia-smi topo -m`

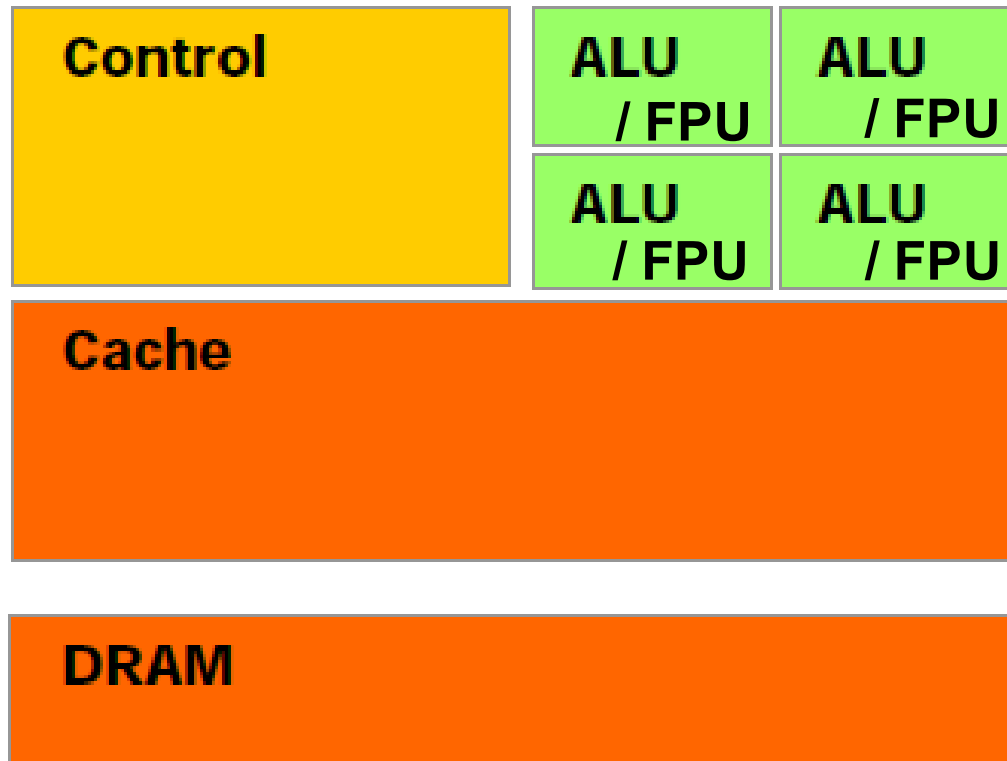
(Schedule a Data Tour with System team)

<https://www.fibermall.com/blog/nvidia-ai-gpu-server-pcie-vs-sxm.htm>

GPU architecture is devoted to massively parallel data processing

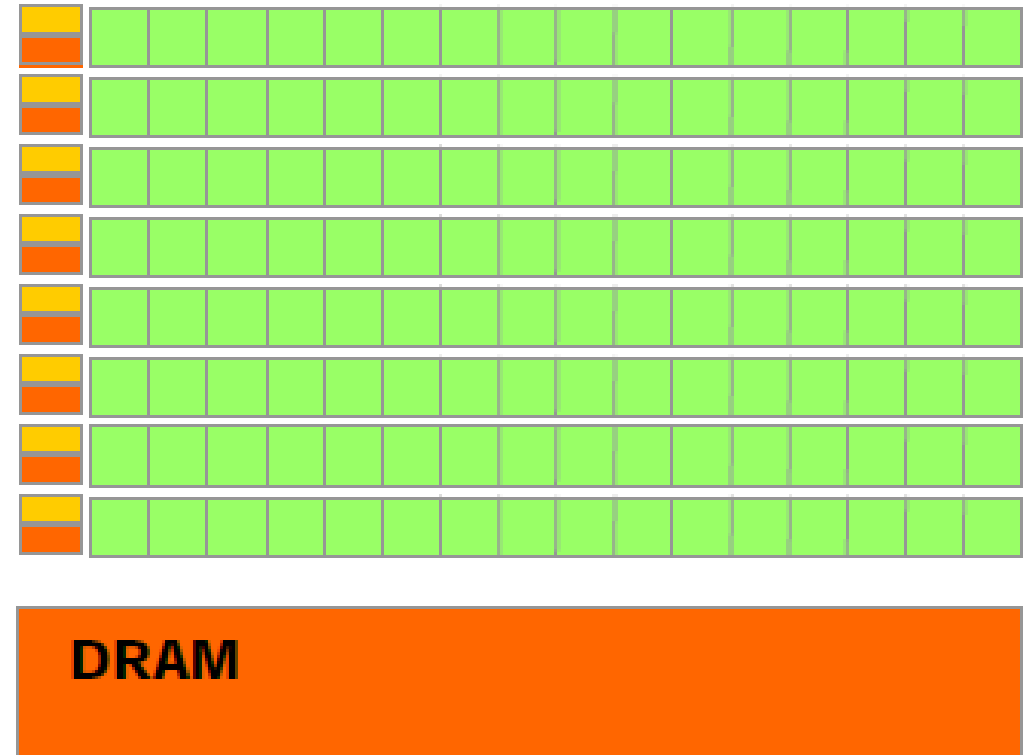
CPU: Focused on individual thread performance

- Large cache per core
- Dedicated units for logic and control



GPU: focused on high throughput

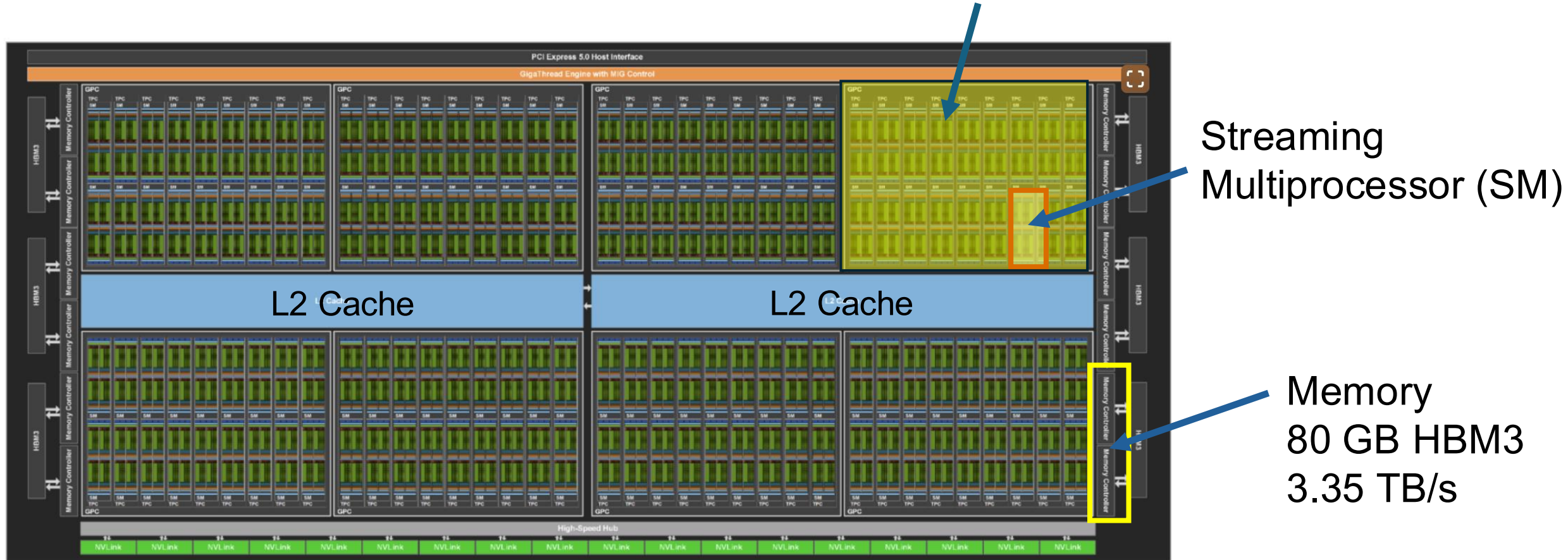
- a lot of ALUs and FPUs, few logic and control units
- thousands of active threads to hide memory latency



ALU = Arithmetic Logic Unit / FPU = Floating-Point Unit

NVIDIA Hopper GH100 (2022)

Graphics Processor Cluster (GPC)

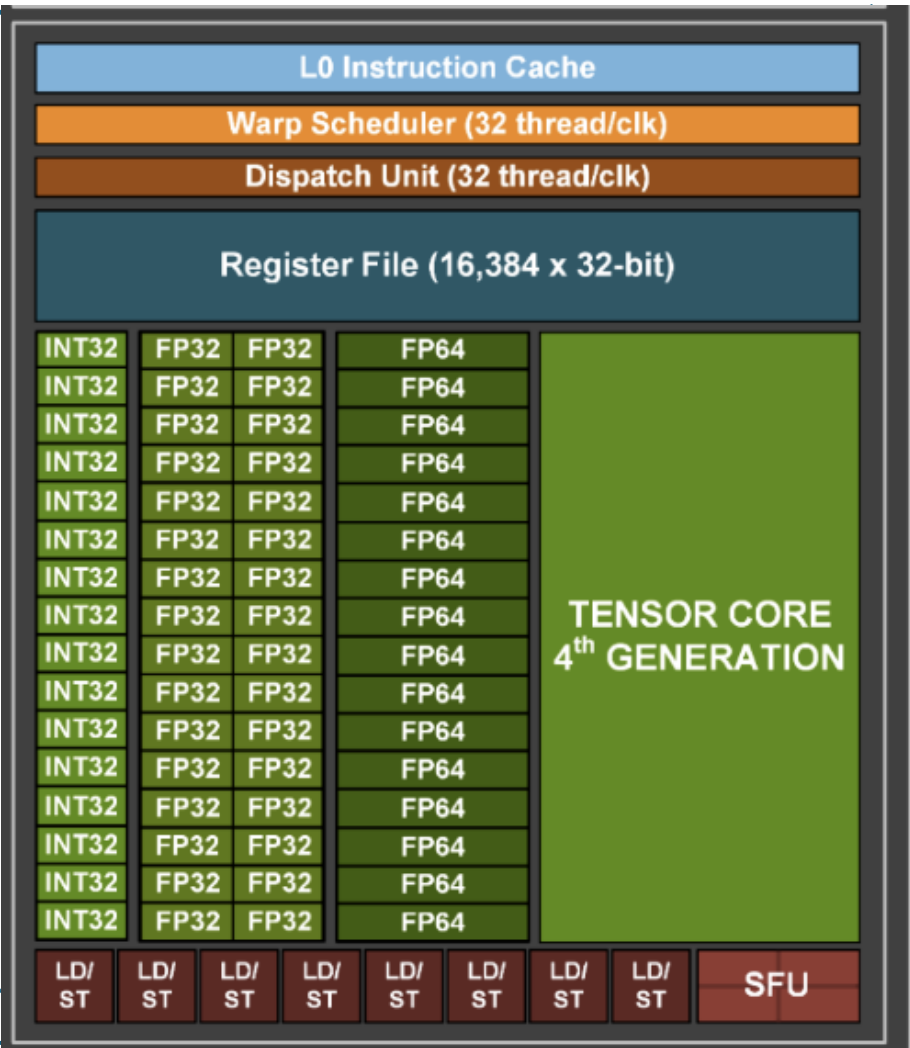
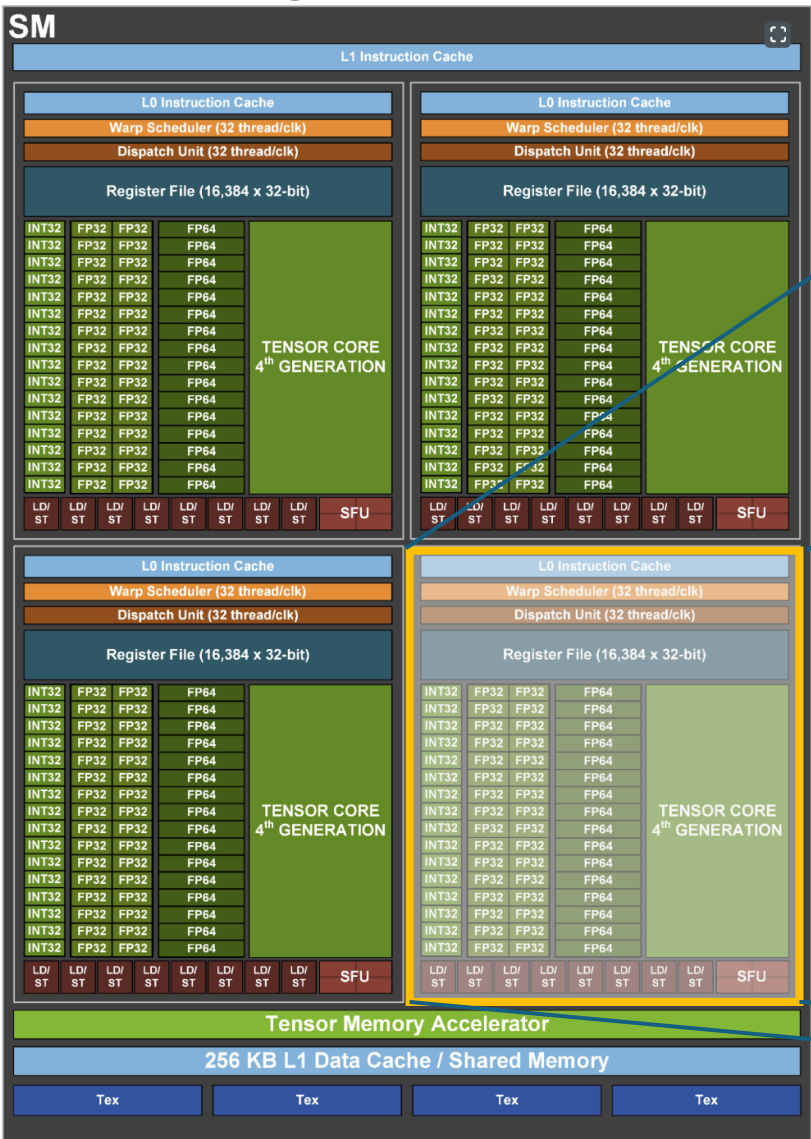


8 GPCs, each with 18 SMs: 114-132 SMs in total

Streaming Multiprocessor (SM)

SIMD = Single Instruction, Multiple Data

32-wide SIMD cluster



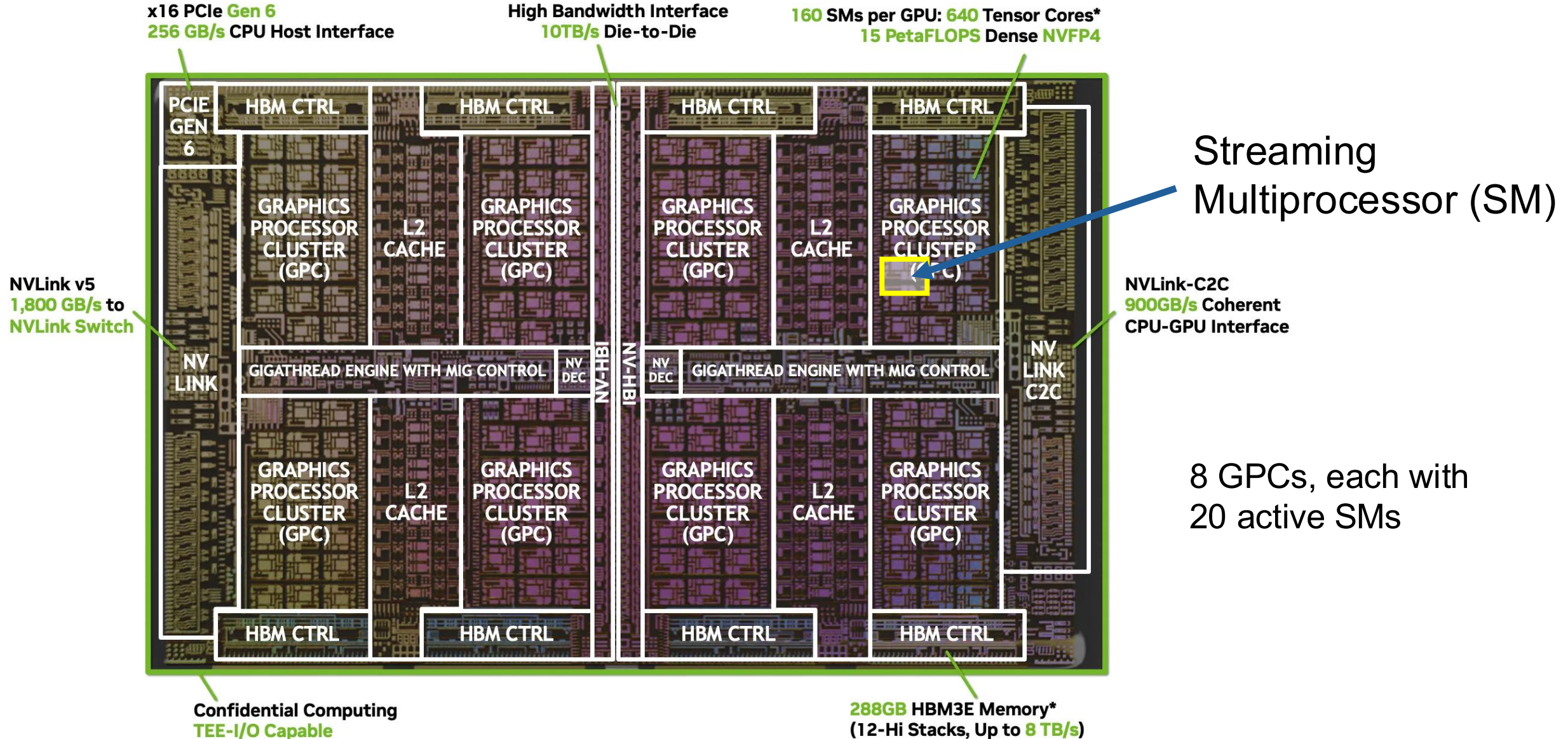
16 ALU 32 FP32 16 FP64

4 SIMD clusters

FP32 = single precision; FP64 = double precision

core clock rate 1.4 GHz
(1.4 x10⁹ clock cycles/sec)

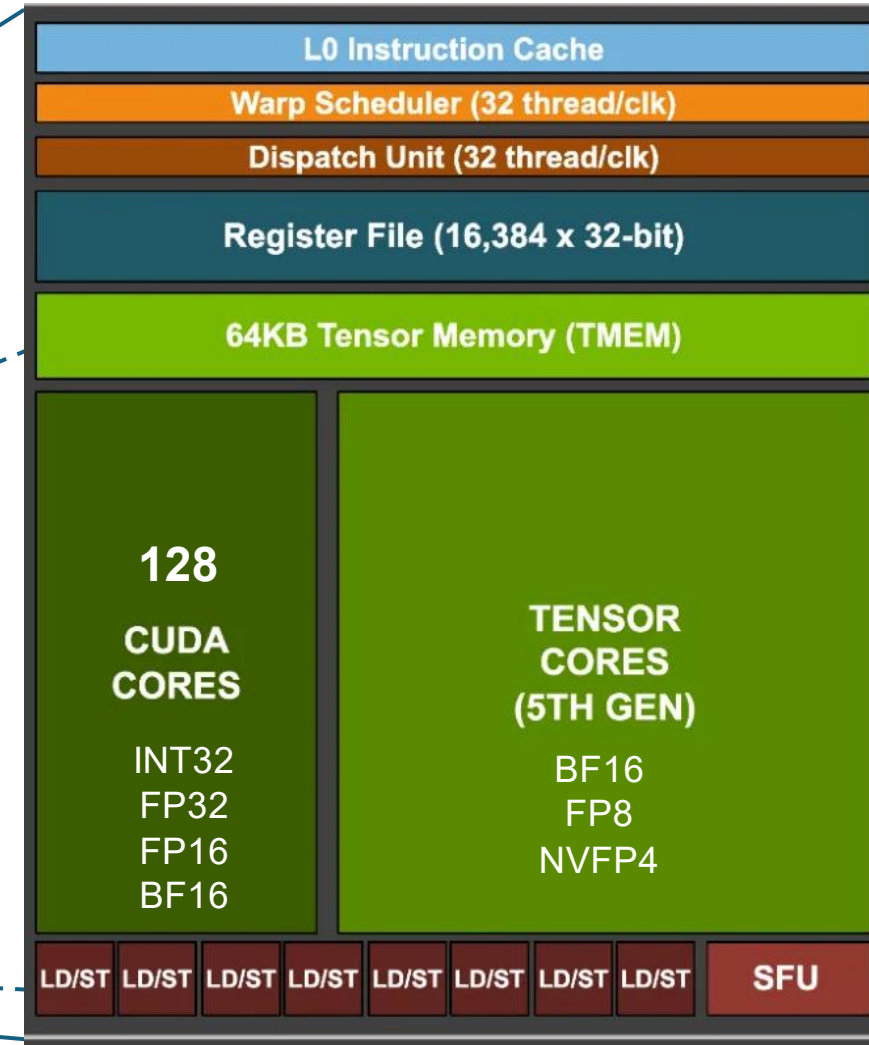
NVIDIA Blackwell B200 (2025): more SMs, bigger memory at a higher bandwidth



More powerful SM

SIMD = Single Instruction, Multiple Data

32-wide SIMD cluster

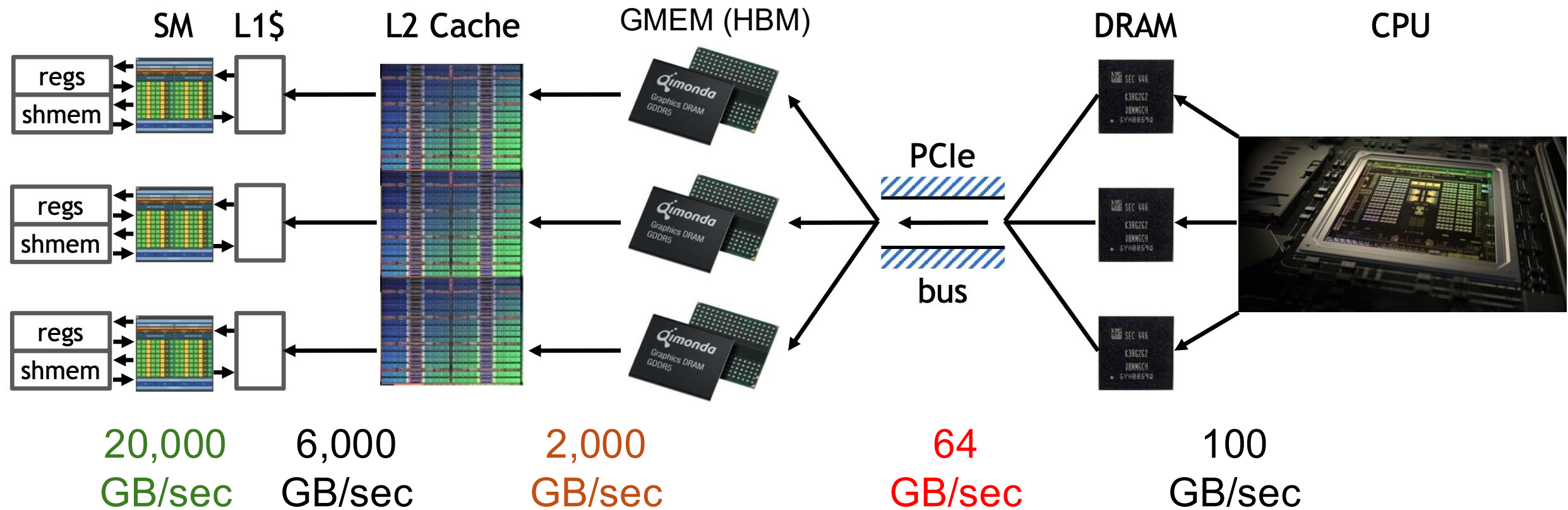


4 SIMD clusters

LDST = Load/Store Units; SFU = Special Function Units

core clock rate ~1.1 GHz
(1.1×10^9 clock cycles/sec)

Device memory: The closer to the cores, the higher bandwidths, the smaller the size



GPU programming model:

Kernels, Memory management, Streams

- Kernel: a sequence of instructions (i.e. a function) to be executed by a GPU thread
 - Load data from memory
 - Perform calculations with the data
 - Write output back to memory
- Kernels are executed by as many GPU threads as needed: SIMD
- Memory allocation on the GPU and copies b/t GPU VRAM and CPU RAM can be done asynchronously with kernel launches
- Stream: a command queue on the GPU that consists of kernel launches and memory copies executed in a specified order (First In First Out)

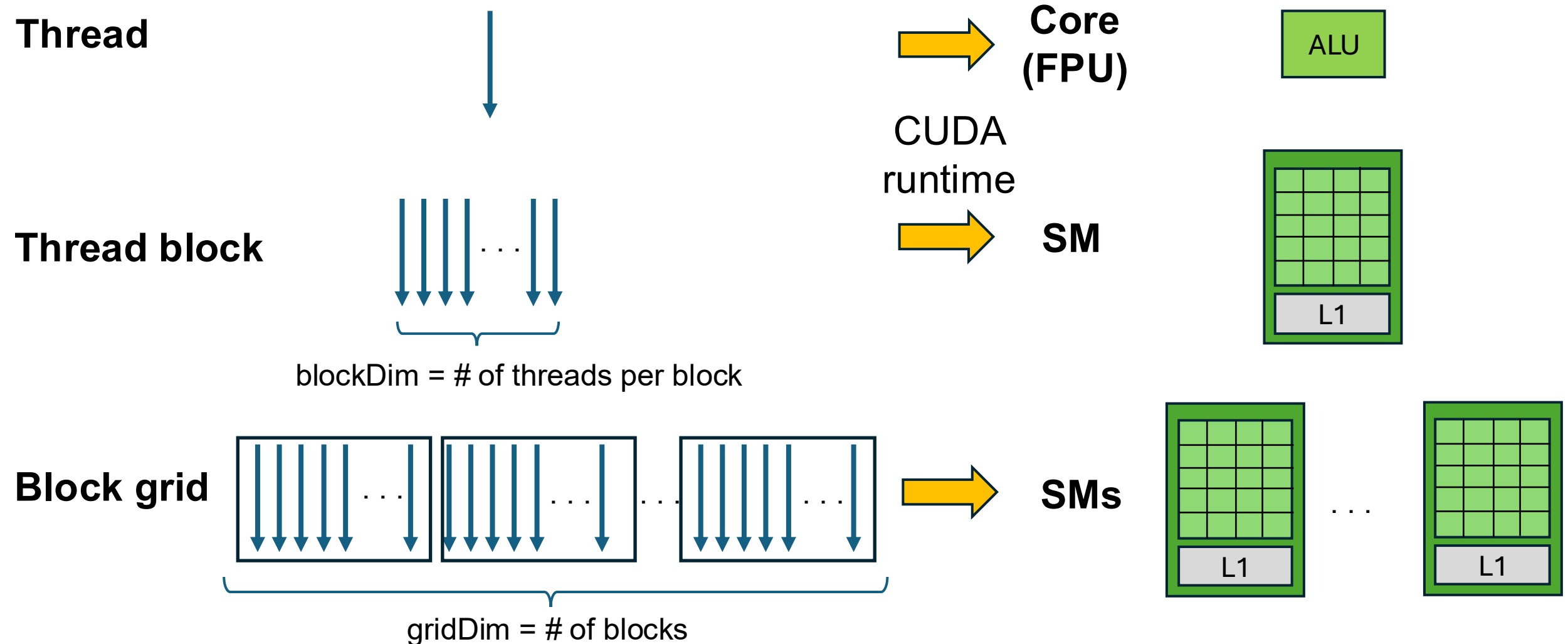
Threads, Thread Blocks and Grid

- Execution of a kernel on the GPU implements the Single-Instruction Multiple-Threads (SIMT) model
 - Thread = the execution unit assigned to a FPU (e.g a CUDA core) to execute the kernel
 - Block = a group of threads to be executed by a SM
 - composed of sets of warps (32 threads) to execute in lock step
 - Grid of thread blocks = all the thread blocks to be mapped to the SMs on the GPU

CUDA runtime executes kernels on hardware depending the specified launch configuration

`kernel_name<gridDim, blockDim, streamID>(arg1, arg2)`

Hardware



NVIDIA NSight: Performance analysis suite

- **NVIDIA NSight Systems:**

- application-level profiling tool
- timeline view of CPUs and GPU actions
- when the kernels run, how compute overlap with memory transfer
- when the GPUs are idle

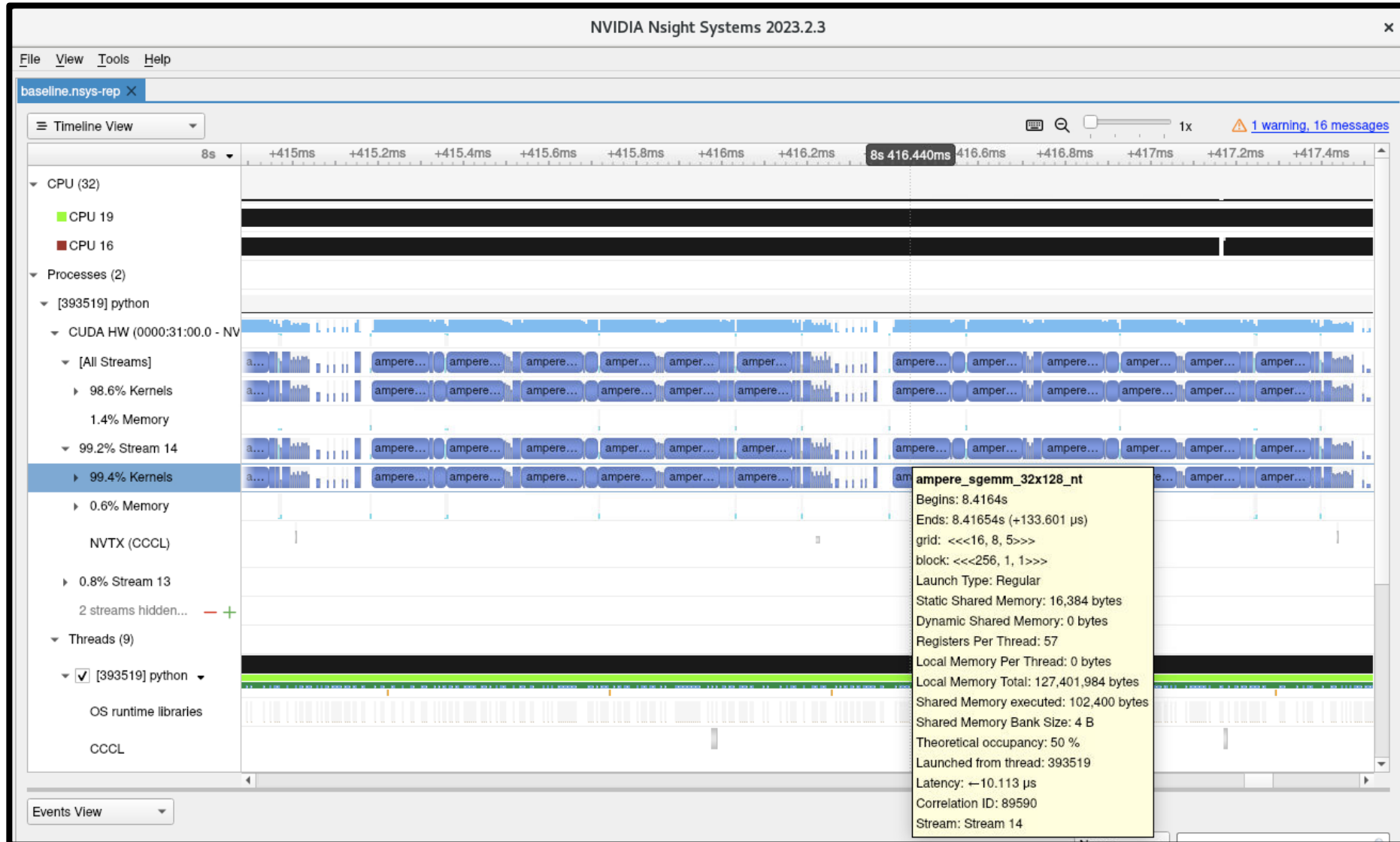
- **NVIDIA NSight Compute:**

- per-process kernel-level profiling tool
- kernel level metrics: roofline charts, memory stores/loads, execution statistics
- need to have `options nvidia NVreg_RestrictProfilingToAdminUsers=0` in `/etc/modprobe.d/nvidia.conf` on the compute node

<https://docs.nvidia.com/nsight-systems/UserGuide/index.html>

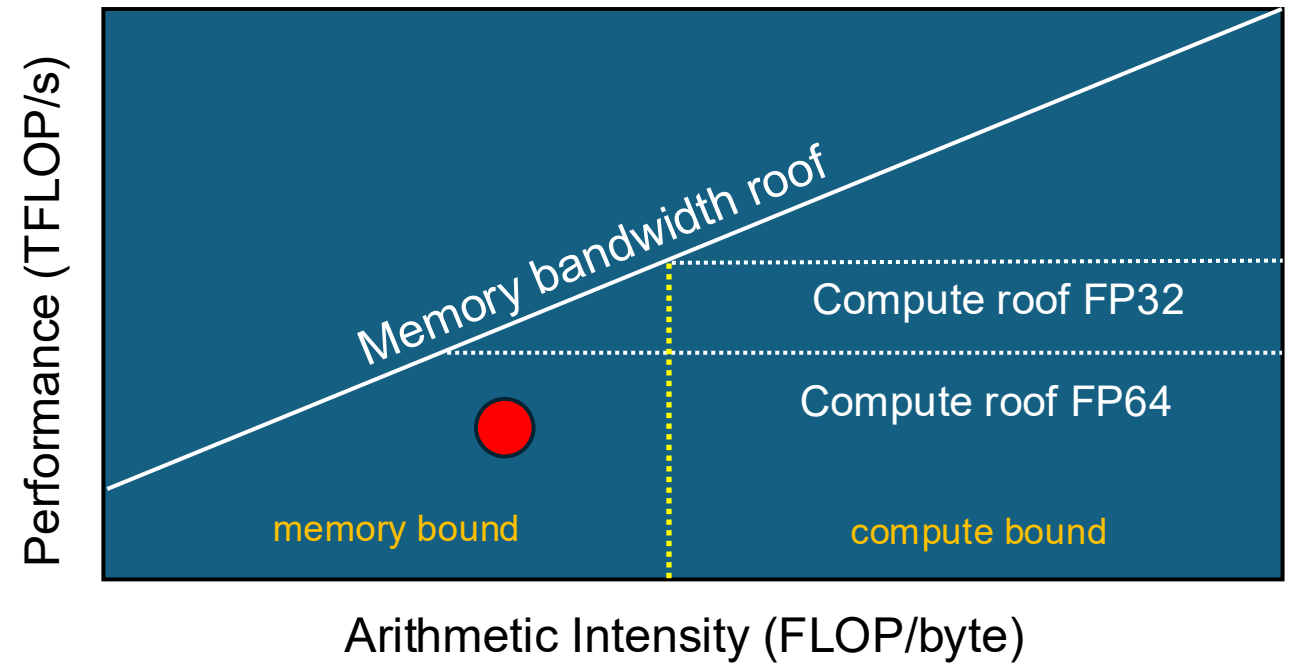
<https://docs.nvidia.com/nsight-compute/NsightCompute/index.html>

Performance depends on CPU and GPU usage, memory transfers and execution patterns



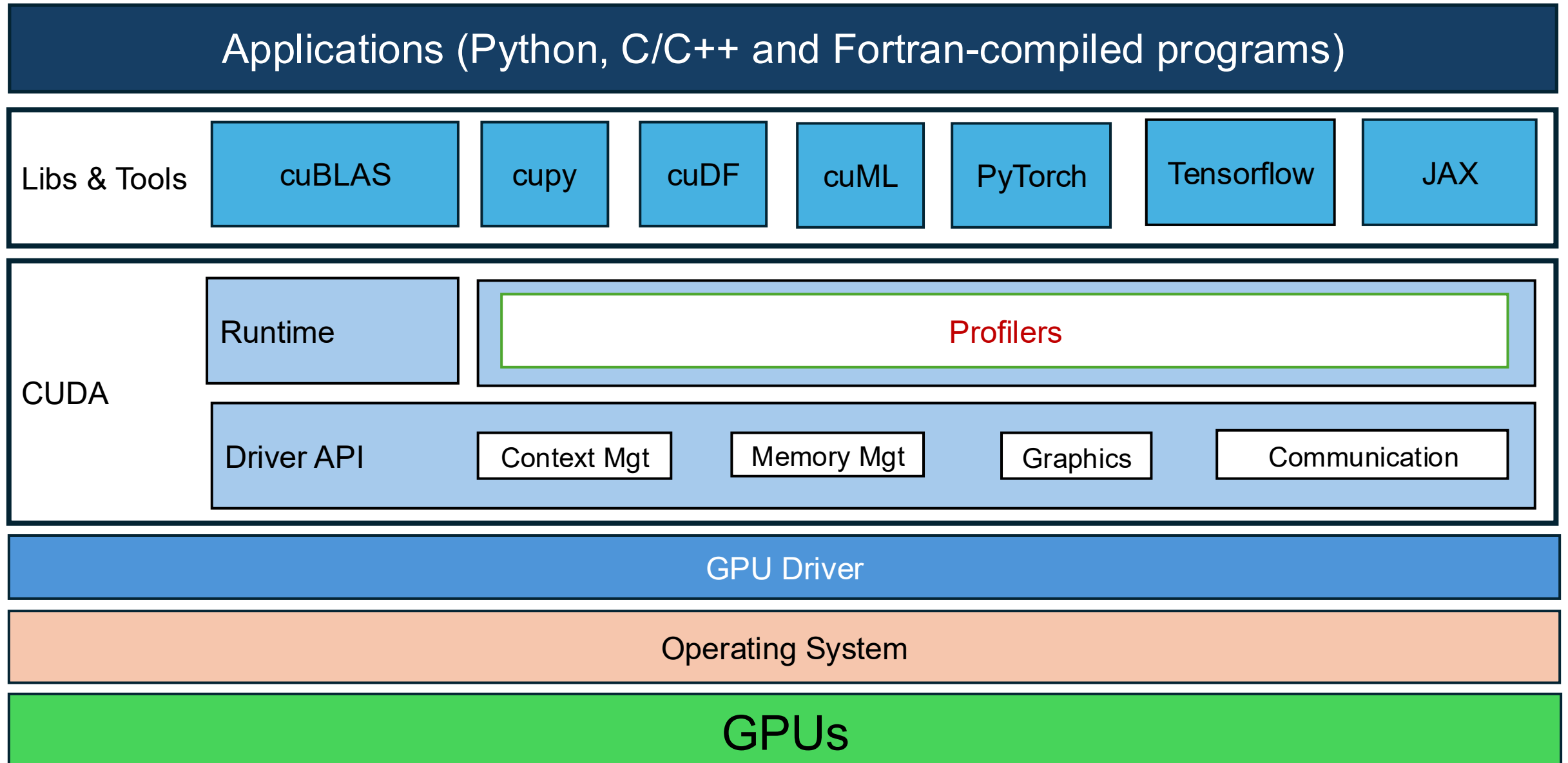
Kernel optimization: Achieved vs theoretical bandwidths

- Roofline charts
 - memory bandwidth boundary
 - compute roof
- Memory access patterns
- Memory transactions
- Kernel launch statistics
- SM occupancy



Goal: The achieved values close to the compute bound roof

The more your code depends on tools and libraries, the fewer knobs you can tune



What constitute an optimized GPU code?

Recommended practices:

1. Minimize the data transfers between CPU RAM and GPU
2. Overlap compute with data transfers and CPU-side tasks
3. Maximize SM utilization: the concurrency of instruction execution on the CUDA cores and Tensor Cores
4. Coalesced memory access: using registers, shared memory and L1 cache, and L2 cache
5. Minimize warp divergence: all the threads in a warp execute the same instruction (not necessarily the same data)
6. Avoid shared memory bank conflicts: threads in a warp do not read from the same bank

On Midway3

- Sign in to the Open OnDemand portal: <https://midway3-ondemand.rcc.uchicago.edu/> under cVPN or on-campus network
- Launch the Midway3 Desktop App
- On the compute node, open a Terminal app:
`module load cuda/12.2 python/miniforge-25.3.0`
- Clone the repo
`git clone https://github.com/ndtrung81/cuda-profiling`

or go to

`cd /project/rcc/shared/cuda-profiling`

Multi-Layer Perceptron (MLP) training

- Run the training code in a GPU job: This is the baseline version that attempts to follow the recommended practices

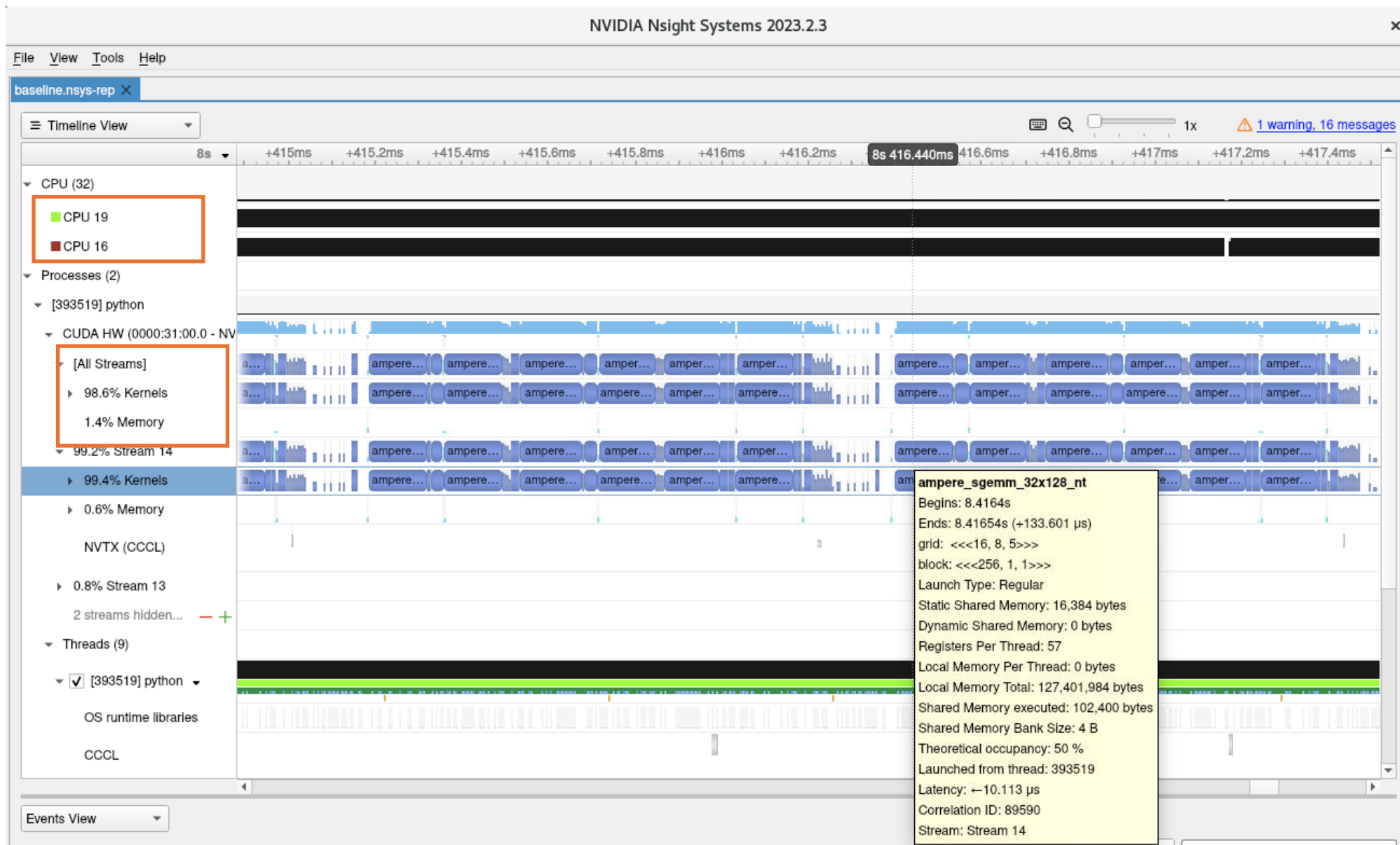
```
python3 ml_training_cuda.py
```

- Profile the run with `nsys` profile

```
nsys profile --trace=cuda,nvtx,cublas,osrt --cuda-memory-usage=true --gpu-metrics-device=all --stats=true -o baseline python ml_training_cuda.py
```

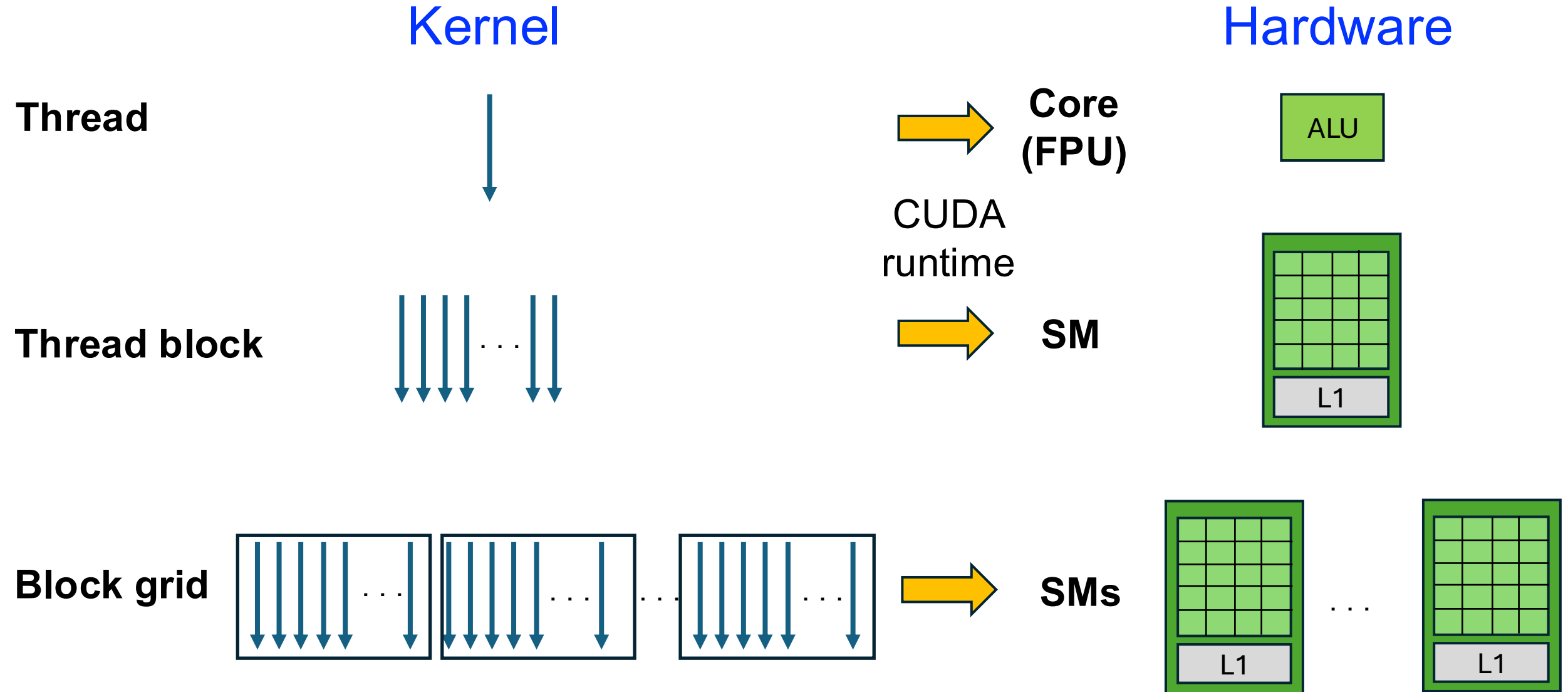
- Show the report with `nsys-ui`

```
nsys-ui baseline.nsys-rep
```

Kernel: name, execution time, execution configuration, memory allocation

Kernels are executed by threads



Profiling kernels with NSight Compute

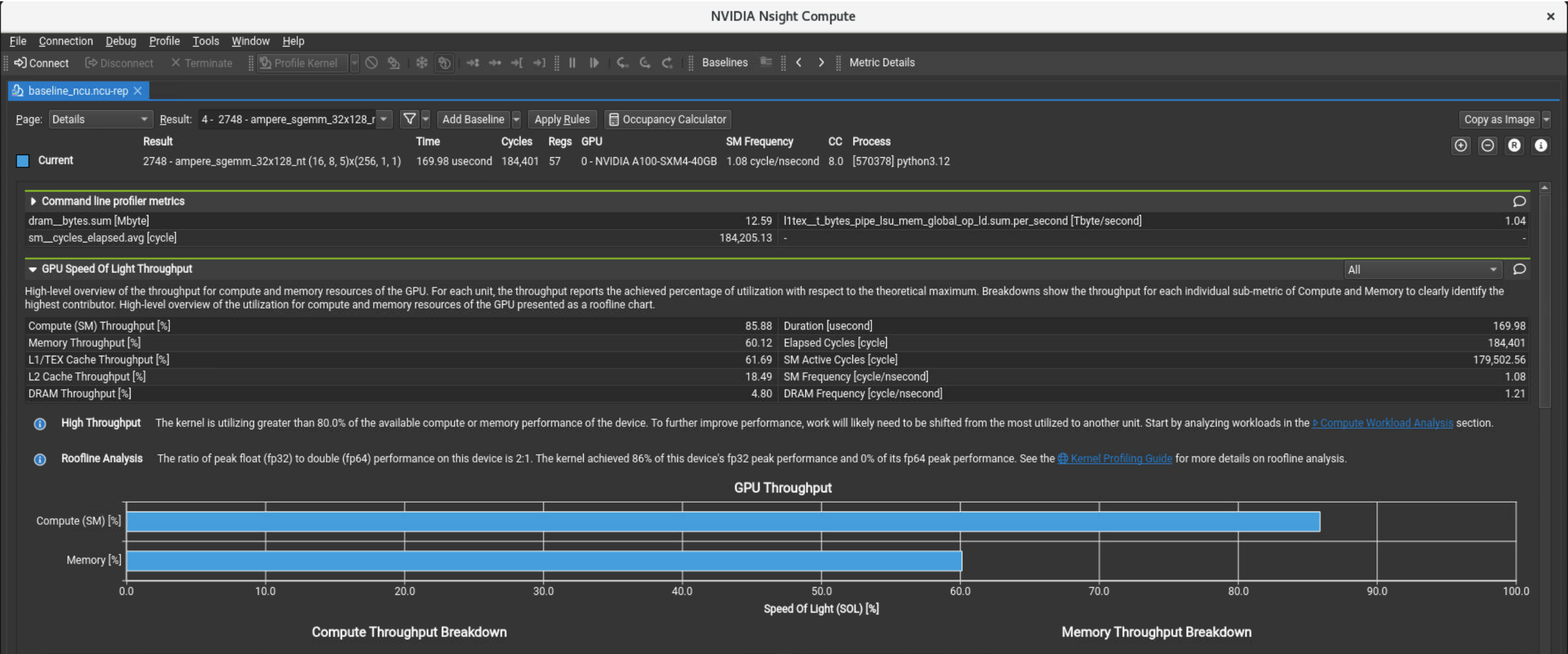
- Profile the run with `ncu`

```
ncu --target-processes all --section LaunchStats --section  
MemoryWorkLoadAnalysis --section Occupancy --section  
SpeedOfLight_RooflineCharts --launch-skip 100 --section Occupancy -o  
baseline python ml_training_cuda.py
```

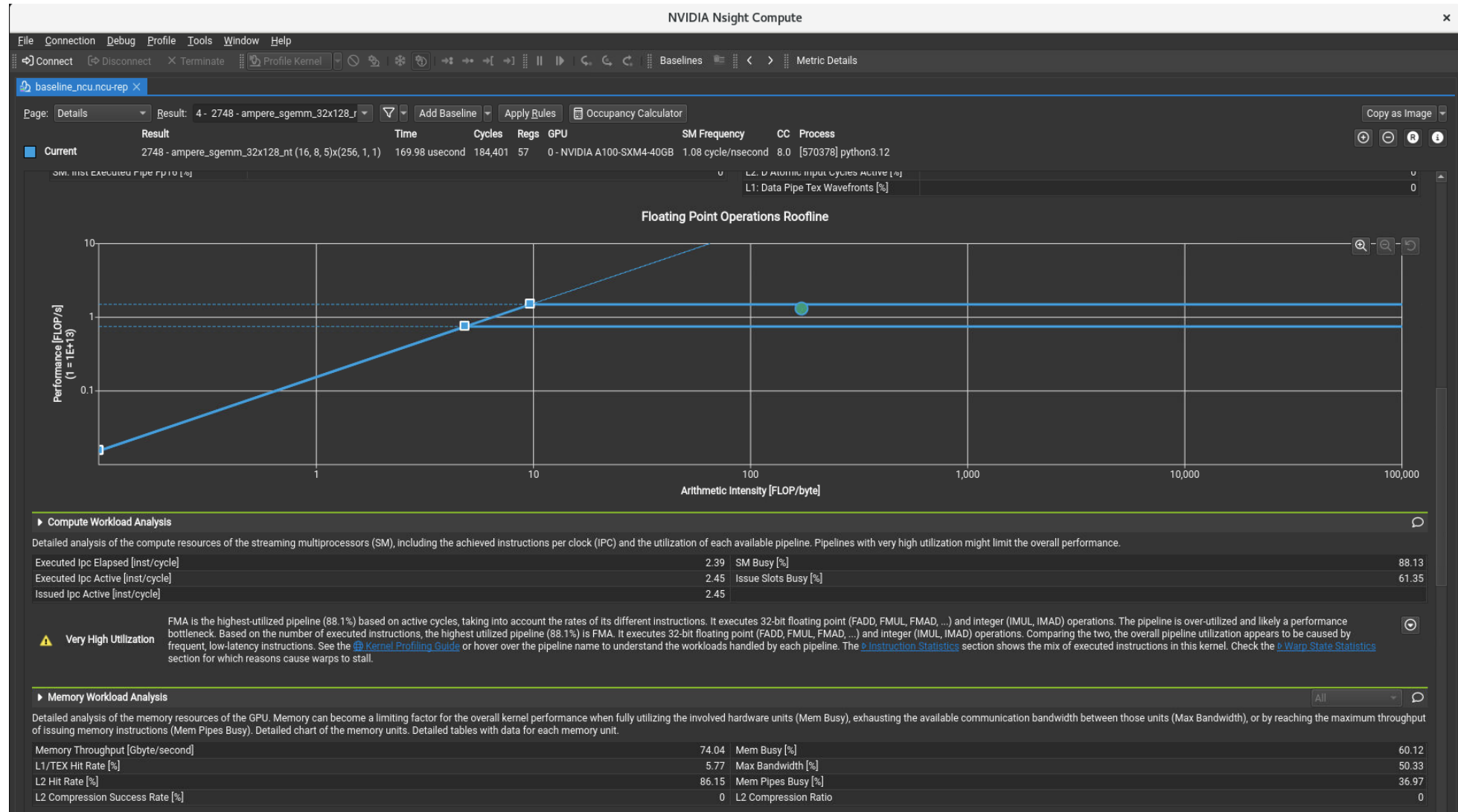
- Show the report with `ncui-ui`

```
ncu-ui baseline.ncu-rep
```

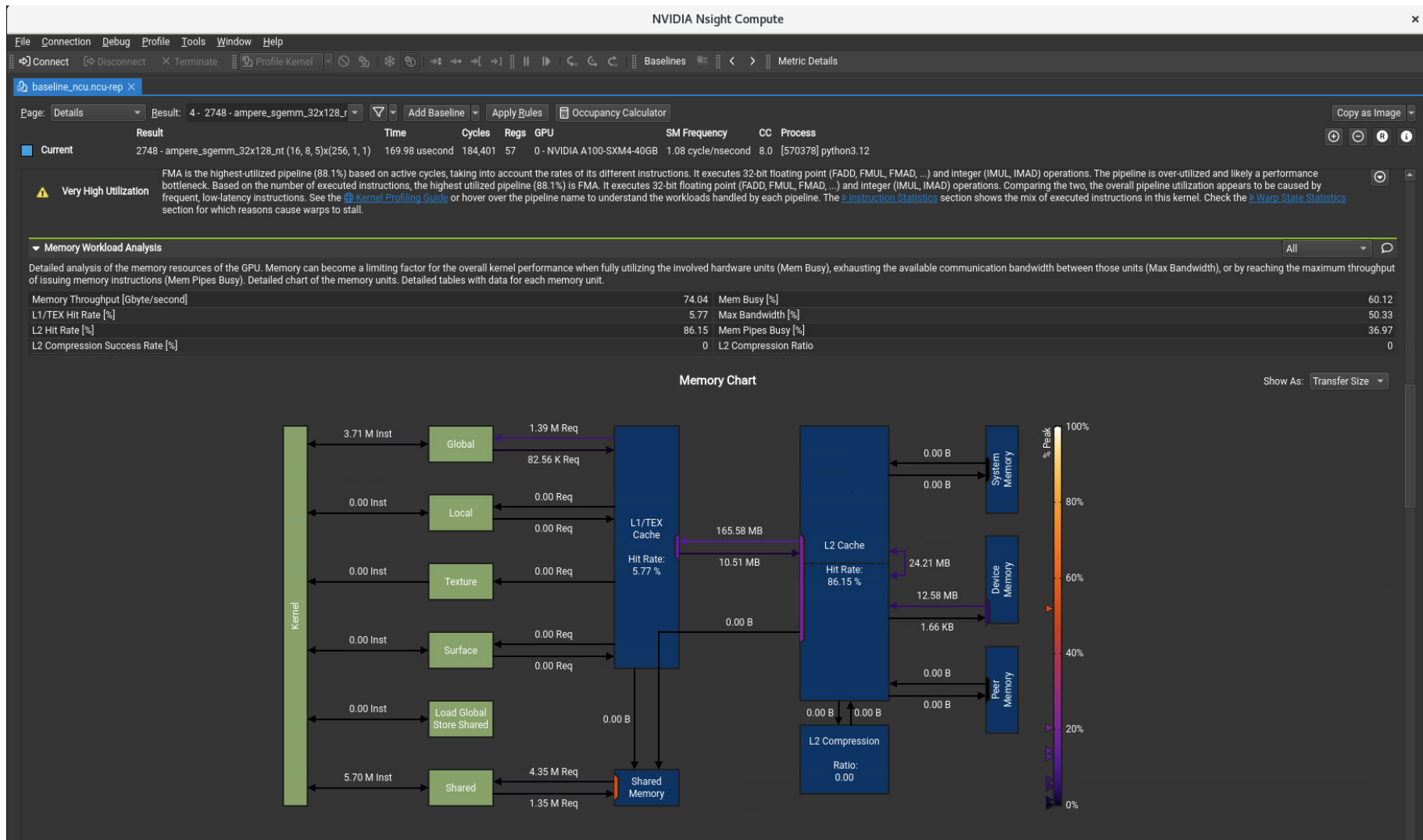
Throughput overview: Compute and Memory



Compute and Memory bandwidth roofs



Memory Workload



Impacts of sub-optimal practices

- I/O gaps (v1)
- **SM underutilization (v2)**
- Without any Tensor Cores activated (v3)
- Roofline gaps (v4)

Running a sub-optimal version

- Run the training code that underutilizes the SMs:

```
python3 ml_training_v2_sm_underutil.py
```

- Profile the run with `nsys profile`

```
nsys profile --trace=--trace=cuda,nvtx,cublas,osrt --cuda-memory-usage=true  
--gpu-metrics-device=all -o v2_sm python ml_training_v2_sm_underutil.py
```

- Show the report with `nsys-ui`

```
nsys-ui v2_sm.nsys-rep
```

What to look at

Speed Of Light

- Compute and Memory throughput

Timeline

- Very short kernel bars — each GEMM finishes in microseconds because only 1–2 SMs are working.
- High ratio of launch overhead to kernel execution time.
- Many more kernel launches per unit time (small work → fast finish → next)

How to improve the SM utilization?

- Increase the batch size

More in README.md at <https://github.com/ndtrung81/cuda-profiling> and in the source code: https://github.com/ndtrung81/cuda-profiling/blob/main/ml_training_v2_sm_underutil.py

Take-home messages

1. GPU architecture is designed for high throughput
 - Number of CUDA cores (FP units), and Tensor Cores
 - Hierarchy of memory bandwidths
2. NSight Systems and NSight Compute
 - application-level timeline: GPU-CPU idle gaps
 - performance metrics: compute vs memory access on chip
 - roofline charts: actual vs theoretical bandwidths
3. GPU performance is maximized when
 - host-device transfers are infrequent or overlapped with compute,
 - GPU cores and Tensor Cores are fully utilized
 - coalesced memory access on the device